Software Engineering Lab Experiment - 4

UML 1.1 Main Foundational 9 UML Diagrams

- **Structural:** Class, Object, Component, Deployment
- **Behavioural:** Use Case, Sequence, Communication, State Machine, Activity

Project: CloudCostCalculator — Serverless AWS cost tracking system + API

UML Report (Software Engineering Lens)

Project Overview (What the system is)

CloudCostCalculator is a serverless AWS cost monitoring + reporting system that turns Cost Explorer billing data into:

- **A live cost API** (for a dashboard UI)
- **A custom CloudWatch metric** (`CloudCostCalculator / MTDSpendUSD`)
- **Budget threshold alarms** (`ccc-mtdspend-prod-ge-50`, `ccc-mtdspend-prod-ge-100`, `ccc-mtdspend-prod-ge-200`)
- **SNS email alerting** via topic: `cloud-cost-calculator-BudgetAlertsTopic-QejtJZ6FnhMF`
- **Automated weekly cost report emails** (SNS-delivered)

Your backend is exposed through API Gateway routes:

- `GET /health`
- `GET /snapshot`
- `GET /mapping`
- `PUT /mapping`

…and the `/mapping` endpoint persists a `mapping.json` file in S3 to translate raw AWS service names into business-friendly labels (ex: “AWS Lambda” → “Serverless”).

Why UML matters here

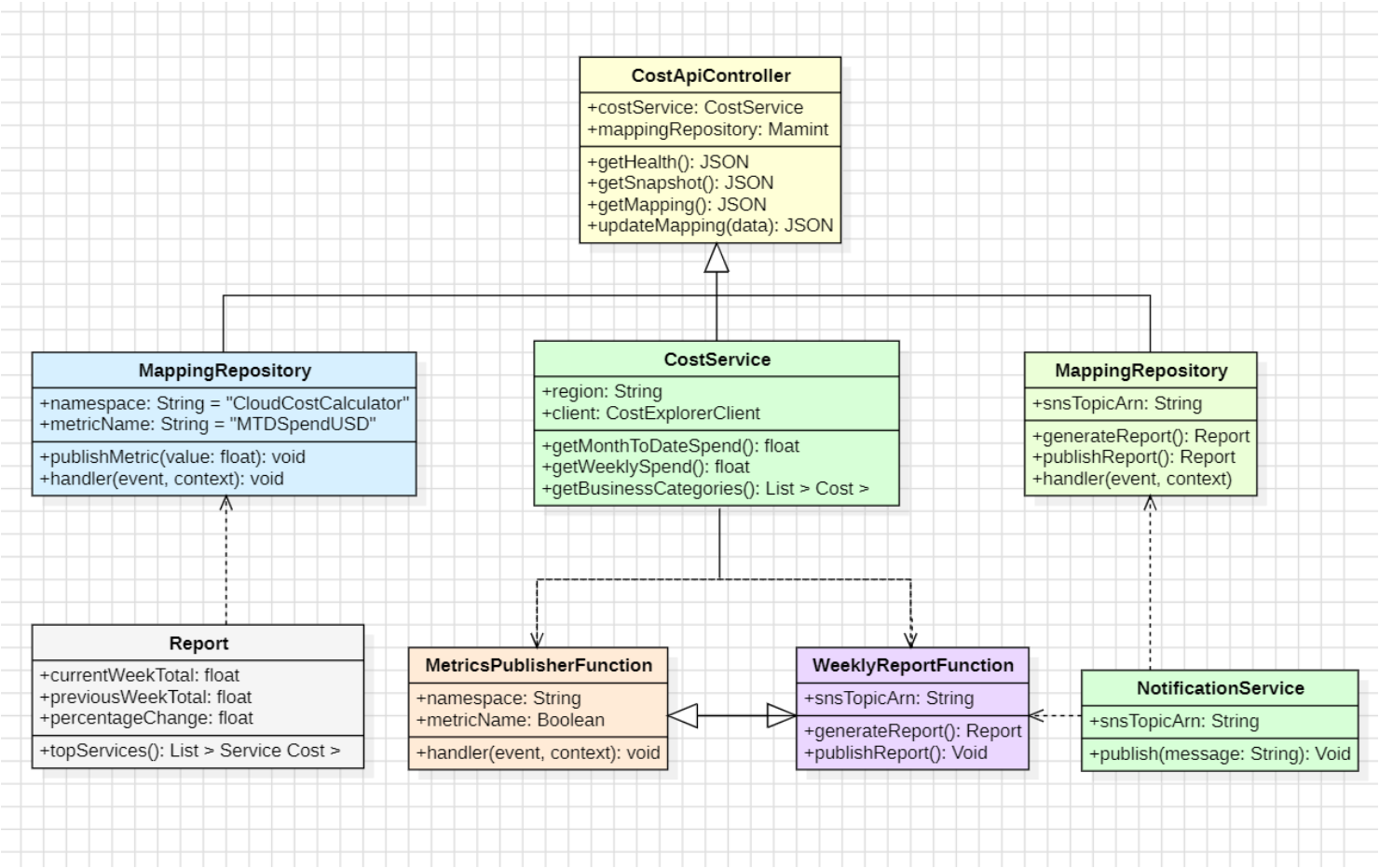
UML helps show your system from **two complementary angles**:

- **Structural UML** (what exists / how it's organized)
    - Class, Object, Component, Deployment
- **Behavioural UML** (how it behaves over time/flows)
    - Use Case, Sequence, Communication, Activity, State Machine

Together, these diagrams prove the project is not just "some AWS glue"—it's a **designed system** with clear boundaries, responsibilities, and runtime behaviour.

STRUCTURAL UML (What the system *is*)

1) Class Diagram



What it depicts

Your **Class diagram** abstracts the core responsibilities into logical units:

- **CostApiController**
  - Represents the API layer behaviour (your `lambda_handler` routing in `backend/app.py`)
  - Methods map directly to routes:
    - `getHealth()`
    - `getSnapshot()`
    - `getMapping()`
    - `updateMapping(data)`
- **CostService**
  - Represents "billing query logic"
  - In your real code, this is essentially:
    - `get_month_to_date_by_service()`
    - `get_week_over_week_totals()`
    - plus any derived calculations
- **MappingRepository / MappingStore**
  - Represents the S3-backed mapping storage:
    - `load_mapping()`
    - `save_mapping()`
  - Tied to `mapping.json` stored in your dashboard S3 bucket.
- **MetricsPublisherFunction**
  - Represents the scheduled publisher that writes:
    - CloudWatch metric: `MTDSpendUSD`
    - Namespace: `CloudCostCalculator`
- **WeeklyReportFunction**
  - Represents the scheduled reporter who composes and publishes the weekly report to SNS.
- **NotificationService**
  - Represents the SNS publish step (alerts + weekly report).

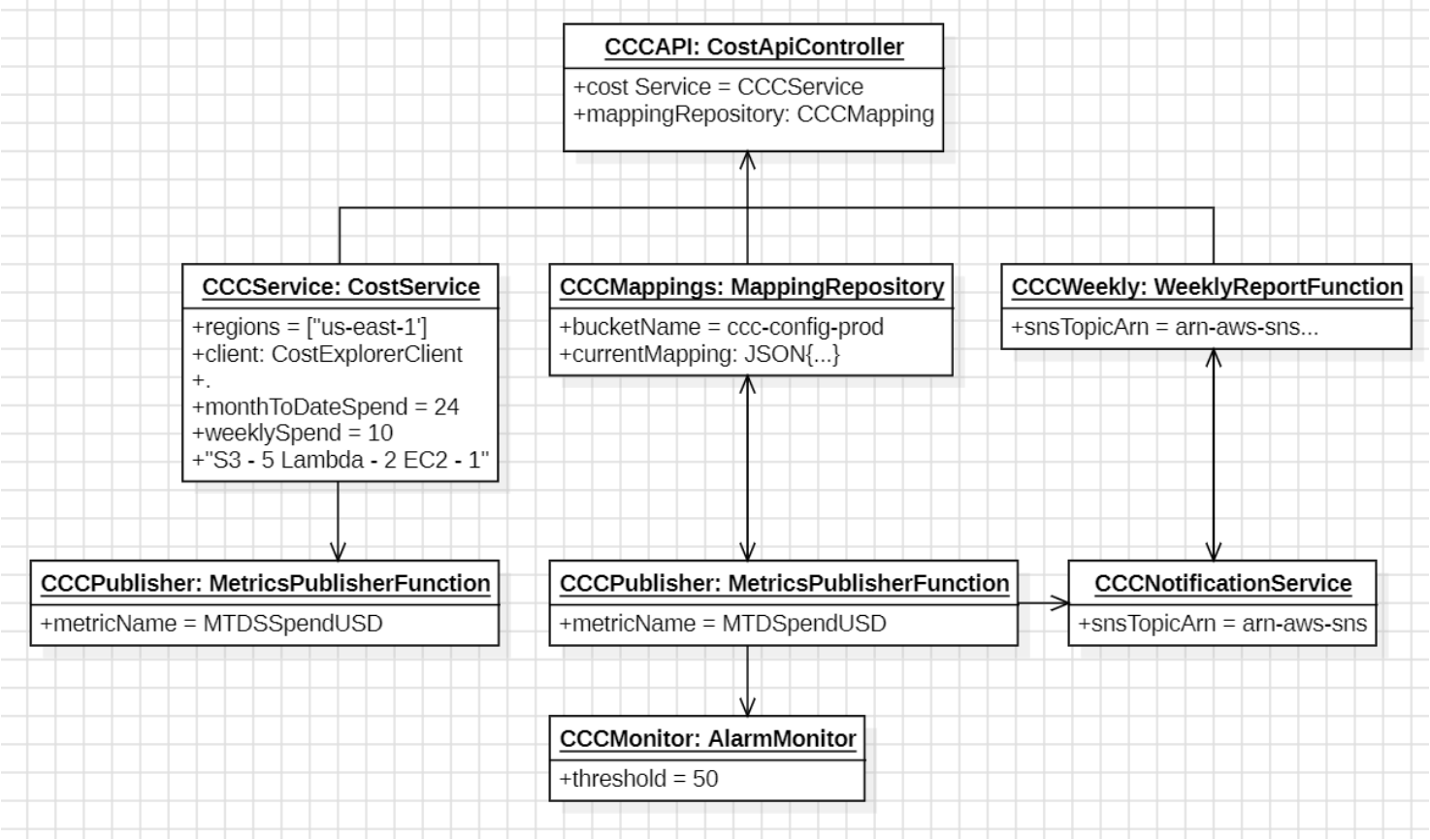
How it reflects your repo + AWS reality

Even though your Python is largely "functional style" inside one Lambda file, the class diagram is still valid as a **design model**:

- Route handlers in `lambda_handler()` behave like controller methods.
- Cost Explorer calls (`ce.get_cost_and_usage`) behave like a "CostService".
- S3 mapping read/write is a clean "repository" boundary.
- SNS publishing is a separate "notification" boundary.
- Scheduled Lambdas are separate application services.

That separation is exactly what recruiters want to see: **clean boundaries, not spaghetti-cloud**.

2) Object Diagram



What it depicts

The **Object diagram** is a “snapshot in time” showing **real instances with real runtime values**, such as:

- A controller instance wired to a service + repository
- A mapping repository instance with:
  - `bucketName = <your dashboard bucket>`
  - `currentMapping = {...}`
- A publisher instance with:
  - `metricName = MTDSpendUSD`
- A “monitor/alarm” object with:
  - `threshold = 50` (and conceptually also 100/200)

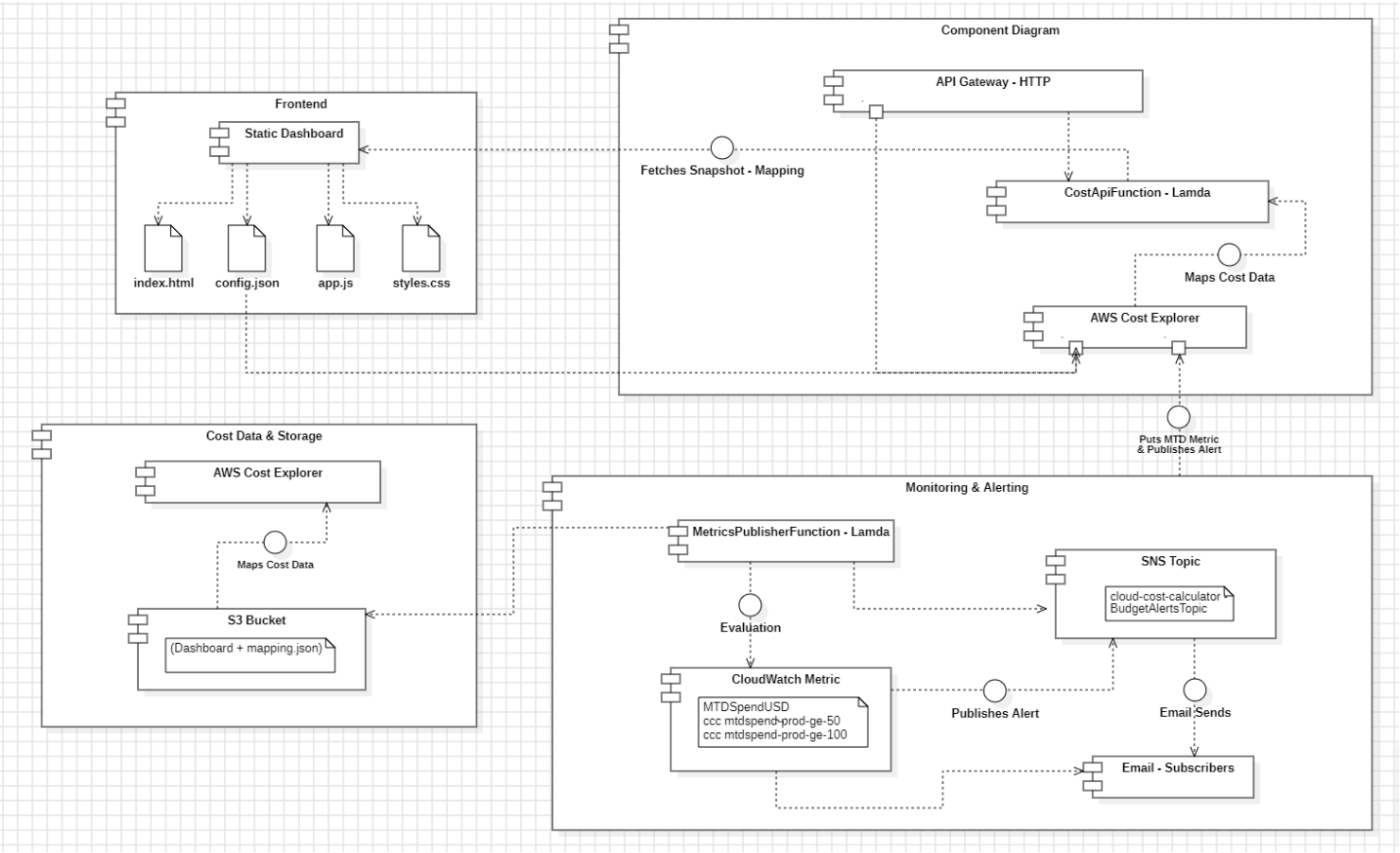
How does it reflect the deployed environment

This diagram is basically your system **running in production**:

- `/snapshot` returns data built from **Cost Explorer results + mapping.json**
- CloudWatch alarms are wired to **your metric + thresholds**
- Weekly report data is computed from **two 7-day windows**

---

3) Component Diagram



What it depicts

The **Component diagram** shows your system as **deployable/replaceable units**:

Frontend

- Static Dashboard (HTML/CSS/JS)
- Uses API Gateway `/snapshot`

API Layer

- API Gateway (HTTP API)
- Integrates with `CostApiFunction` (Lambda)

Data & Mapping

- Cost Explorer (billing source of truth)
- S3 bucket (hosts dashboard assets + `mapping.json`)

Monitoring & Alerting

- `MetricsPublisherFunction` → CloudWatch Metric (`MTDSpendUSD`)
- CloudWatch Alarms (`ccc-mtdspend-prod-ge-*`)
- SNS Topic (`cloud-cost-calculator-BudgetAlertsTopic-QejtJZ6FnhMF`)
- Email Subscribers (confirmed)

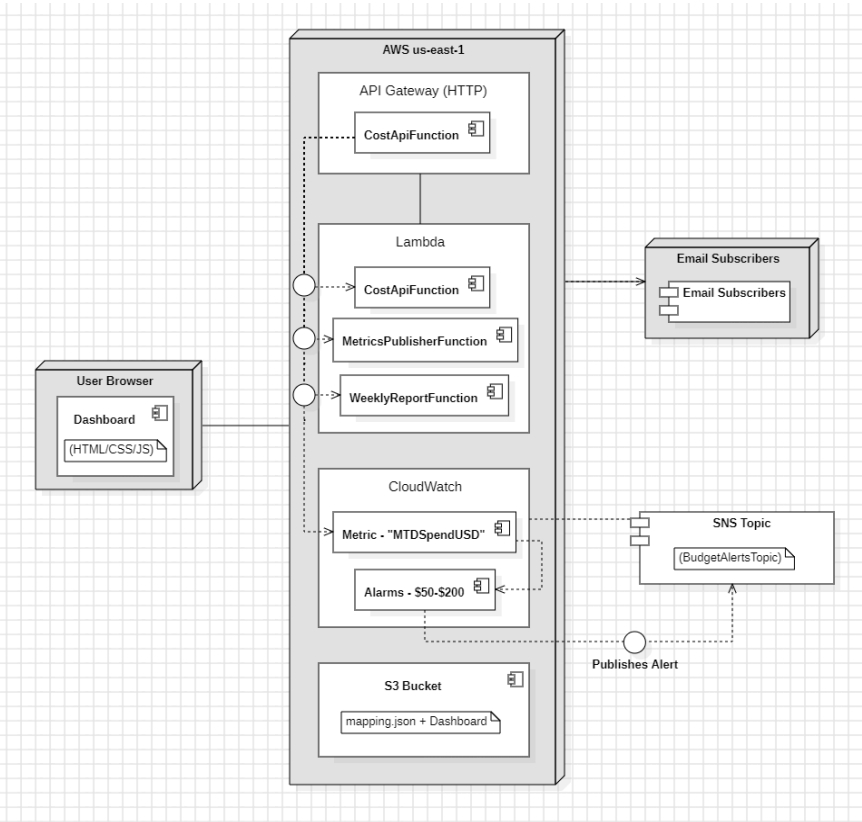
How does it reflect your actual AWS wiring

This diagram matches your real screenshots and naming:

- The alarms are explicitly tied to `MTDSpendUSD`
- SNS topic + email subscription is confirmed
- API Gateway routes are Lambda-integrated
- Dashboard is separate and consumes API results

---

4) Deployment Diagram



What it depicts

The **Deployment diagram** shows “where things run”:

- **User Browser**
  - Loads the static dashboard
- **AWS us-east-1**
  - API Gateway (HTTP)
  - Lambda:
    - Cost API function
    - MetricsPublisherFunction
    - WeeklyReportFunction
  - CloudWatch:
    - Metric `MTDSpendUSD`
    - Alarms ($50/$100/$200)
  - S3 bucket:
    - `mapping.json` + dashboard files
- **SNS Topic**
  - Publishes alerts & weekly summaries
- **Email Subscribers**
  - Receive messages

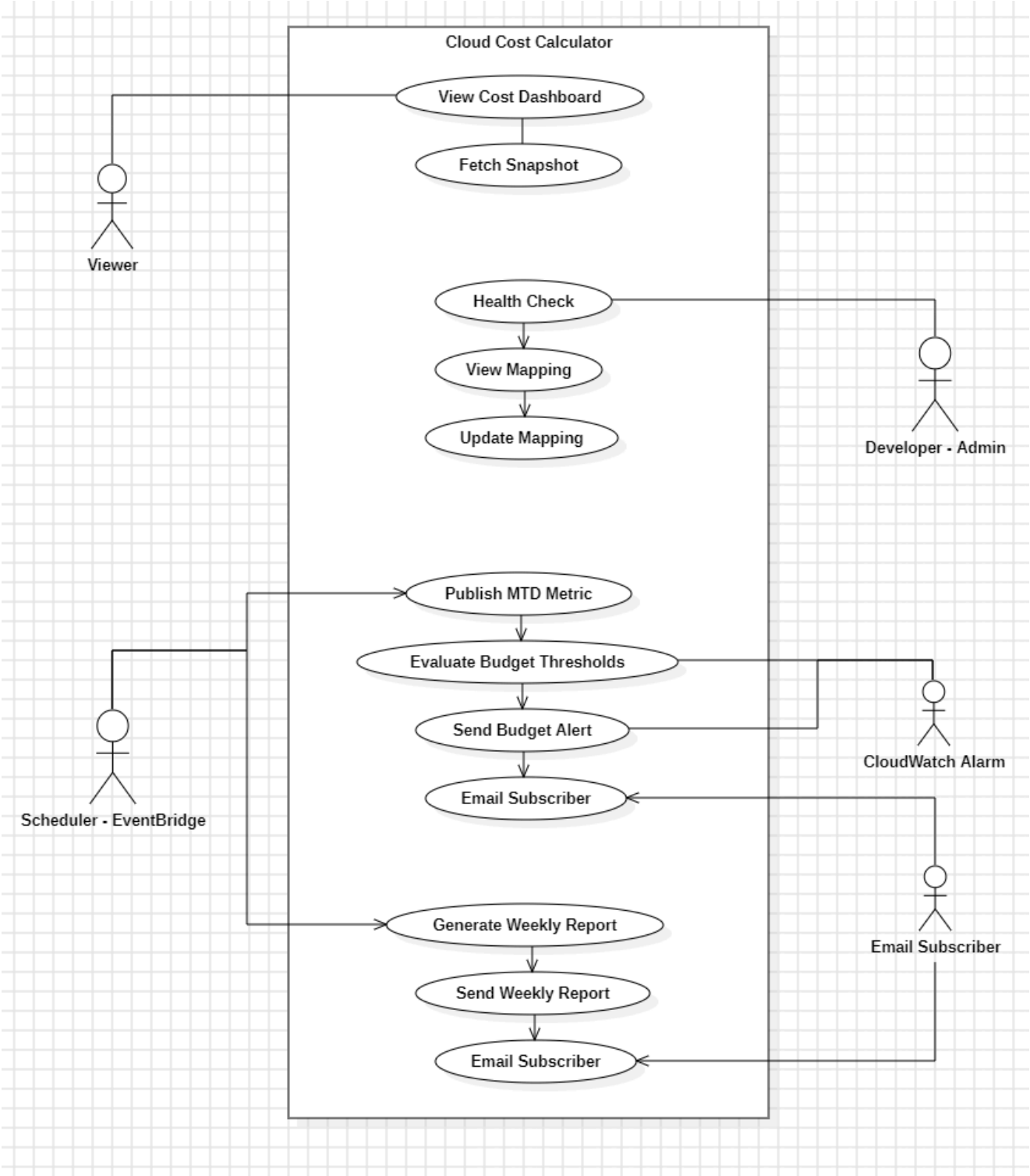
Why this matters (software engineering lens)

Deployment diagrams prove you understand:

- runtime boundaries (browser vs AWS)
- managed services vs compute
- event-driven signals (alarms → SNS)
- that “serverless” is still a real deployment topology

BEHAVIORAL UML (what system *does over time*)

5) Use Case Diagram



What it depicts

Your Use Case diagram captures the system goals from the perspective of actors:

Actors

- **Viewer**: wants to see the cost dashboard
- **Developer/Admin**: wants to manage mappings, verify health
- **Scheduler (EventBridge)**: triggers automation
- **CloudWatch Alarm**: triggers alerts
- **Email Subscriber**: receives notifications

Use cases mapped directly to your project

- View Cost Dashboard → (Fetch Snapshot)
- Health Check → `GET /health`
- View Mapping → `GET /mapping`
- Update Mapping → `PUT /mapping` (protected by `x-admin-token` if set)
- Publish MTD Metric → scheduled MetricsPublisherFunction
- Evaluate Budget Thresholds → CloudWatch Alarm evaluation
- Send Budget Alert → SNS publish
- Generate Weekly Report → scheduled WeeklyReportFunction
- Send Weekly Report → SNS email delivery

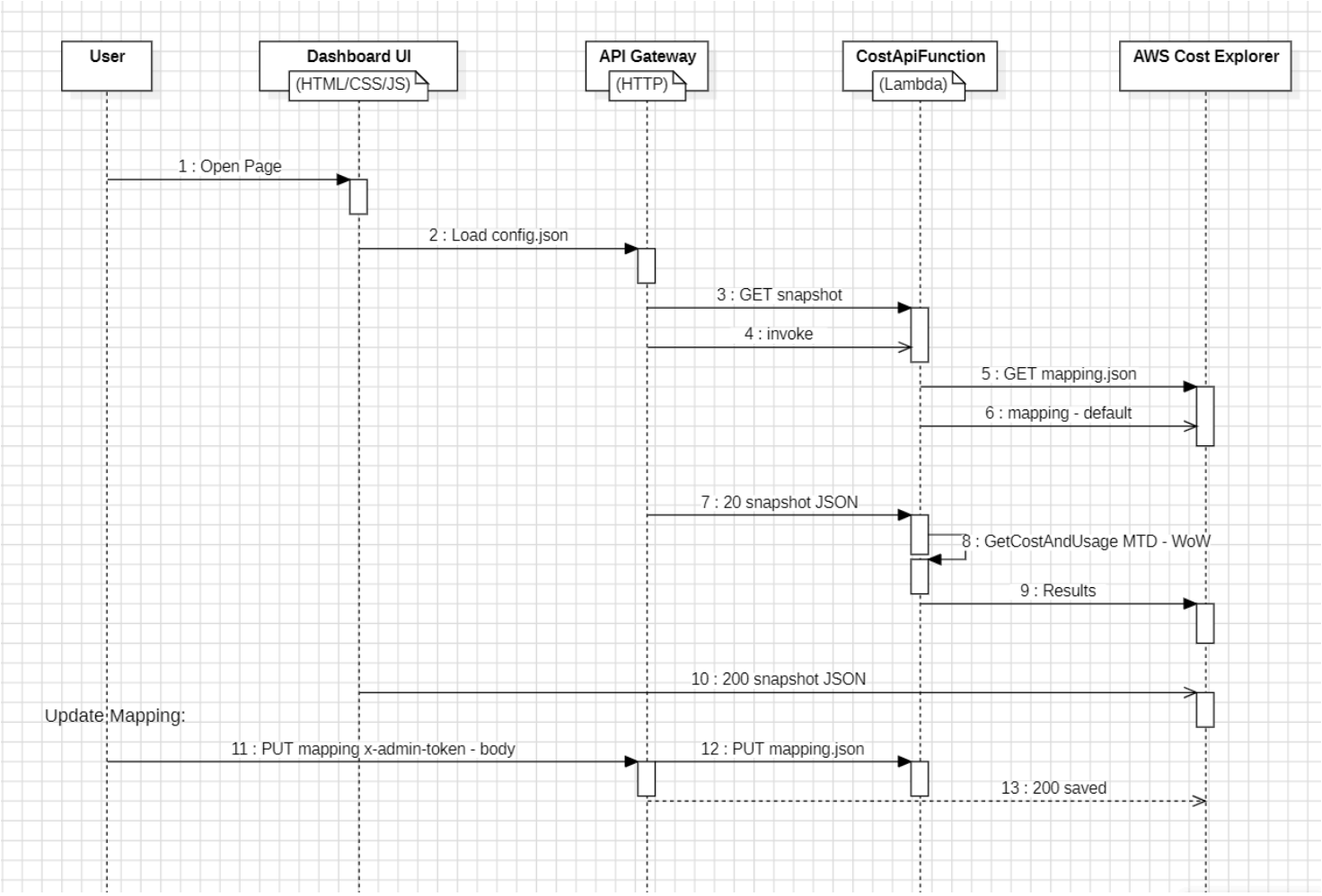
Why it's accurate

It matches exactly what your system does in production:

- People interact via dashboard + API
- Admin modifies mapping for clean business labelling
- Automation runs without humans
- Alarms + email subscribers close the loop

---

6) Sequence Diagram



What it depicts

The **Sequence diagram** shows the time-ordered flow for your most important runtime path:
Dashboard → API → Lambda → AWS APIs → Response

Typical flow (matches your diagram):

1. User opens the dashboard
2. Dashboard loads config (often `config.json`)
3. Dashboard calls `GET /snapshot`
4. API Gateway invokes `CostApiFunction`
5. Lambda loads `mapping.json` from S3 (or defaults if missing)
6. Lambda calls Cost Explorer:
    - MTD by service
    - WoW totals
7. Lambda returns snapshot JSON
8. Dashboard renders cards/charts from JSON

It also includes the admin path:

- `PUT /mapping` with `x-admin-token` (if configured)
- Saves to S3 and returns “200 saved”

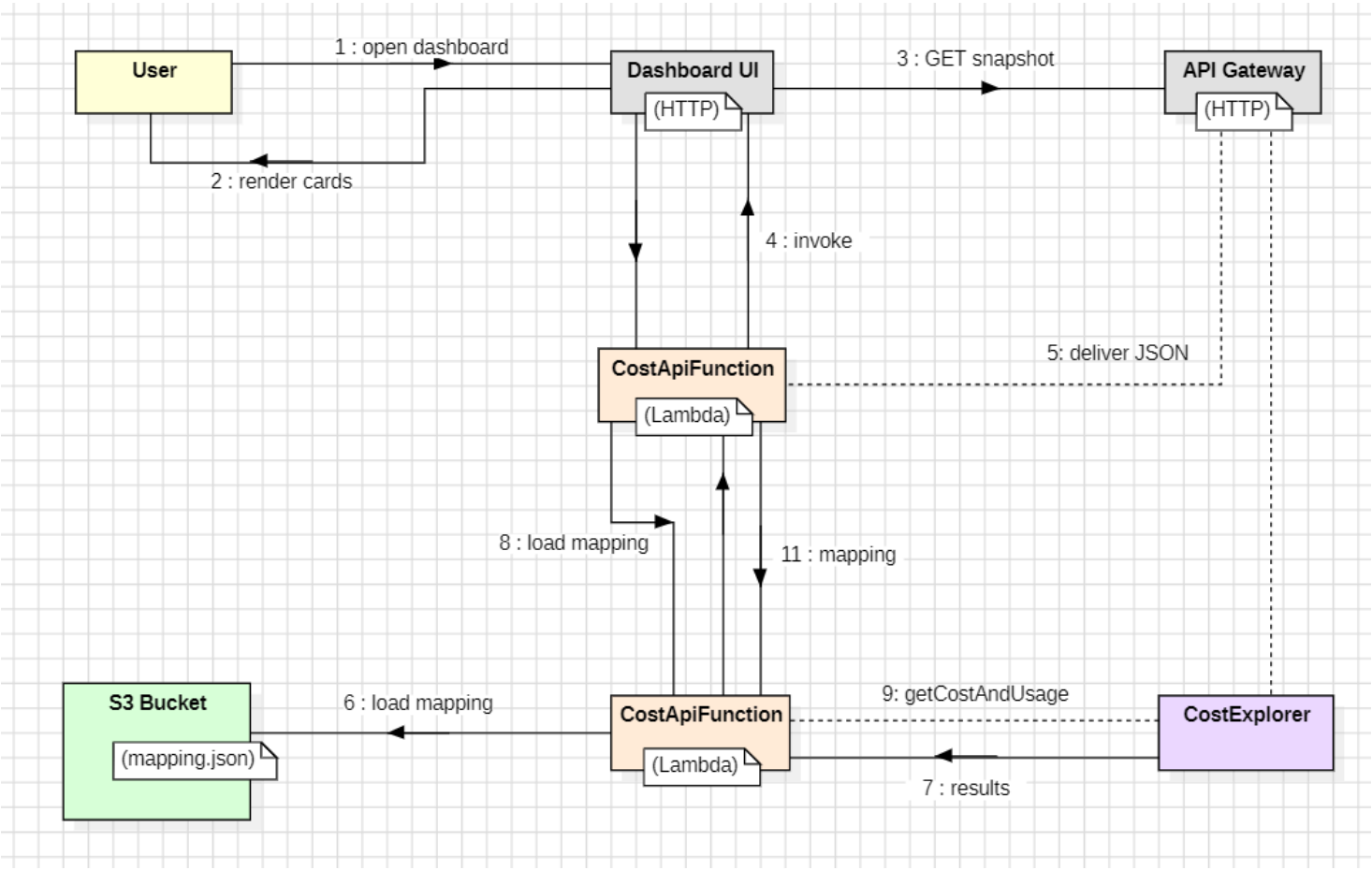
Why it matters

Sequence diagrams show you understand:

- request lifecycles
- integration boundaries
- where latency comes from (Cost Explorer calls)
- and how the state is persisted (S3 mapping)

---

7) Communication Diagram



What it depicts

The Communication diagram is basically the sequence diagram's cousin:

- same participants
- but emphasizes **who talks to whom** rather than exact vertical time

It highlights:

- Dashboard UI communicates with API Gateway
- API Gateway routes to Lambda
- Lambda communicates with:
    - S3 (mapping)
    - Cost Explorer (billing)
- Response returns to UI for rendering

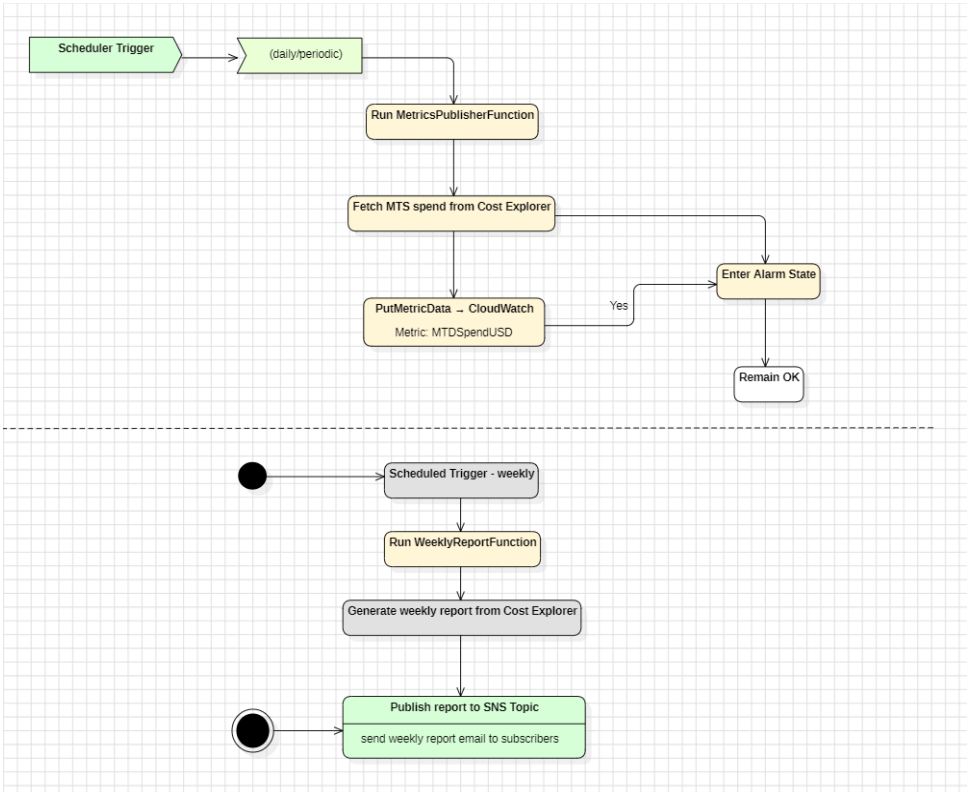
Why would you include both

In software engineering documentation:

- Sequence = best for "step-by-step runtime story"
- Communication = best for "interaction topology/coupling"

---

8) Activity Diagram



What it depicts

Your Activity diagram shows **workflow logic** for two major automated pipelines:

A) Daily/Periodic Metrics Pipeline

- Scheduler trigger (EventBridge)
- Run `MetricsPublisherFunction-qHICp98n9xt3`
- Fetch MTD spend from Cost Explorer
- Publish `MTDSpendUSD` to CloudWatch
- Decision:
    - If threshold breached → alarm transitions
    - Else remains OK

B) Weekly Reporting Pipeline

- Weekly scheduler trigger
- Run WeeklyReportFunction
- Query Cost Explorer for:
    - current week total
    - previous week total
    - delta/percent change
    - top services + categories
- Publish report to SNS topic
- Email subscribers receive a summary

Why does it reflect your system perfectly

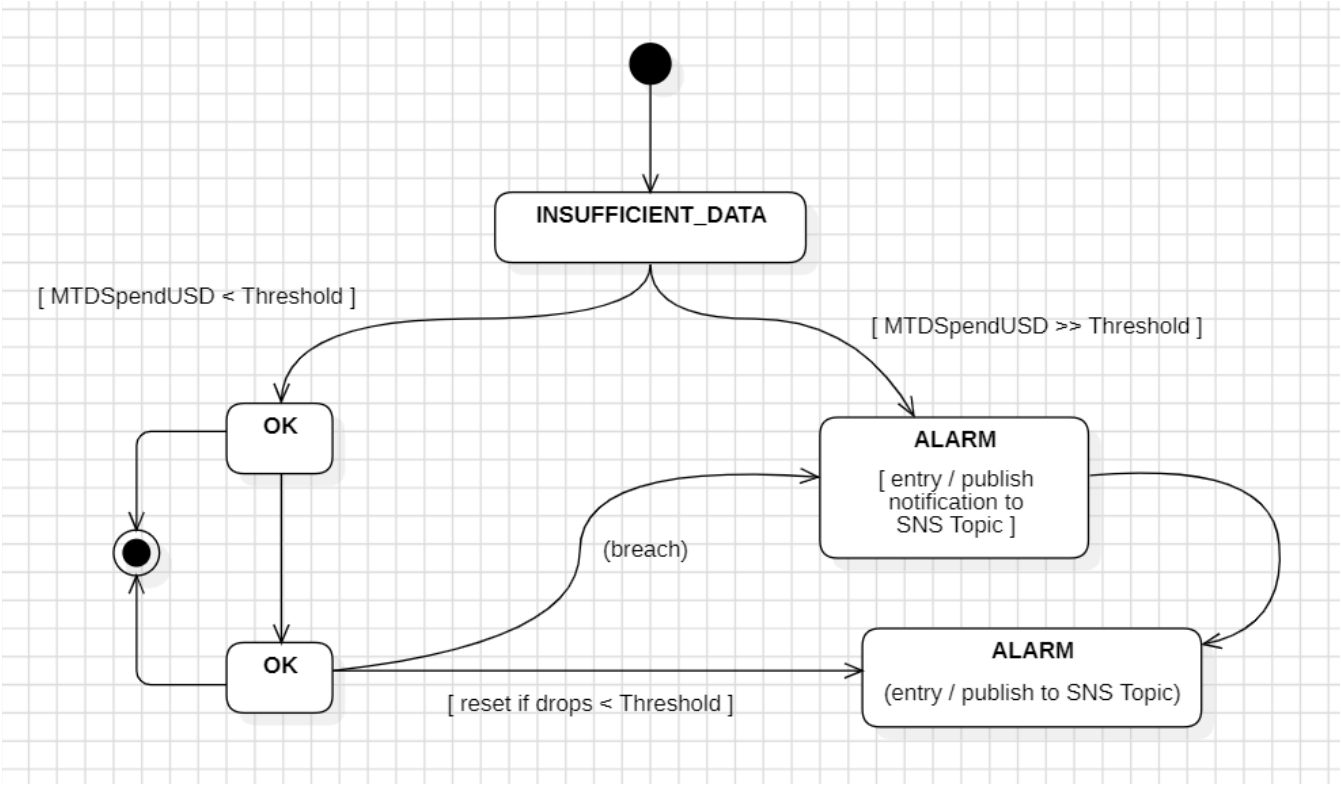
Your project is fundamentally **workflow-driven**:

- scheduled triggers
- data fetch
- metric publish
- alarm evaluation
- notifications

Activity diagrams capture that “business process” view cleanly.

---

9) State Machine Diagram



What it depicts

This diagram models the **CloudWatch Alarm lifecycle** around your metric:

States:

- **INSUFFICIENT_DATA**
- **OK**
- **ALARM**

Transitions are driven by:

- `MTDSpendUSD < threshold` → OK
- `MTDSpendUSD >= threshold` → ALARM (entry action: publish notification to SNS topic)
- "reset if drops < threshold" → OK

How it maps to your real alarms

Your alarms:

- `ccc-mtdspend-prod-ge-50`
- `ccc-mtdspend-prod-ge-100`
- `ccc-mtdspend-prod-ge-200`

All evaluate the same underlying metric and differ by threshold.
So this single state machine model represents the alarm behaviour pattern repeated across thresholds.

Why it matters (engineering view)

This shows:

- You understand **reactive systems**
- Alerting is not “send email every time”
- It’s a stateful evaluation system with transitions + actions

---

Final Engineering Takeaways

Collectively, your UML set demonstrates:

- **Clear layering**
  - UI → API → Lambda → AWS APIs → UI
- **Separation of concerns**
  - mapping persistence (S3)
  - cost retrieval (Cost Explorer)
  - alerting (CloudWatch + SNS)
  - reporting (weekly automation)
- **Event-driven architecture**
  - scheduled triggers
  - metric-driven alarms
- **Production realism**
  - real AWS resource names
  - confirmed SNS subscription
  - actual API routes
  - actual alarm conditions